MySQL Isolation Levels: Complete Guide

Table of Contents

1. Fundamentals
2. The Four Isolation Levels
3. Concurrency Problems
4. Dirty Reads
5. Non-Repeatable Reads
6. Phantom Reads
7. Lost Updates
8. Implementation & Examples
9. Monitoring & Troubleshooting
10. Best Practices

Fundamentals

What are Isolation Levels?

Isolation levels define how transactions interact with each other in a multi-user database environment. They control the trade-off between:

- **Consistency**: Data remains accurate and valid
- **Concurrency**: Multiple users can work simultaneously
- **Performance**: System responsiveness and throughput

ACID Properties

Isolation is the "I" in ACID:

- **Atomicity**: All or nothing
- **Consistency**: Valid state to valid state
- **Isolation**: Transactions don't interfere
- **Durability**: Committed data persists

Why Isolation Matters

Without proper isolation:

- Users see inconsistent data
- Business logic breaks
- Financial calculations fail
- Inventory counts become incorrect

The Four Isolation Levels

1. READ UNCOMMITTED (Isolation Level 0)

Definition: Transactions can read uncommitted (dirty) data from other transactions.

Characteristics:

- Lowest isolation, highest concurrency
- Allows dirty reads, non-repeatable reads, phantom reads
- Rarely used in production
- Faster but most dangerous

MySQL Implementation:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
-- Or set globally
SET GLOBAL TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

Lock Behavior:

- Write locks: NOT released until commit
- Read locks: NOT acquired (no locks)
- Other transactions can read uncommitted writes

2. READ COMMITTED (Isolation Level 1)

Definition: Transactions can only read committed data. Avoids dirty reads.

Characteristics:

- Oracle database default
- Prevents dirty reads
- Allows non-repeatable reads and phantom reads
- Better balance than READ UNCOMMITTED

MySQL Implementation:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Lock Behavior:

- Write locks: Held until commit
- Read locks: Acquired and released immediately
- Phantom reads still possible
- Non-repeatable reads possible

3. REPEATABLE READ (Isolation Level 2)

Definition: Transactions see a consistent snapshot. Prevents dirty and non-repeatable reads.

Characteristics:

- MySQL default
- Uses MVCC (Multi-Version Concurrency Control)
- Prevents dirty reads and non-repeatable reads
- Phantom reads still possible (in some implementations)
- Good balance

MySQL Implementation:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SET GLOBAL TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

Lock Behavior:

- Read locks: Held until commit (prevents other changes)

- Write locks: Held until commit
- Gap locks: Prevent phantom reads in range queries
- InnoDB uses MVCC for non-locking reads

4. SERIALIZABLE (Isolation Level 3)

Definition: Transactions are serialized. Highest isolation, lowest concurrency.

Characteristics:

- Highest isolation level
- Prevents all anomalies
- Equivalent to executing transactions one-by-one
- Significant performance impact
- Use only when absolutely necessary

MySQL Implementation:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Lock Behavior:

- All locks acquired and held until commit
- Read locks: Held for duration
- Write locks: Held for duration
- Shared locks on ranges

Concurrency Problems

Problem Matrix

Problem	READ UNCOMMITTED	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Dirty Read	✓ (Possible)	✗ (Prevented)	✗ (Prevented)	✗ (Prevented)

Problem	READ UNCOMMITTED	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Non-Repeatable Read	✓ (Possible)	✓ (Possible)	✗ (Prevented)	✗ (Prevented)
Phantom Read	✓ (Possible)	✓ (Possible)	✓ (Possible)*	✗ (Prevented)
Lost Update	✓ (Possible)	✓ (Possible)	✓ (Possible)	✗ (Prevented)

*InnoDB with REPEATABLE READ uses gap locks, preventing phantom reads

Dirty Reads

Problem Definition

A **dirty read** occurs when a transaction reads data written by an uncommitted transaction. If the writing transaction rolls back, the reading transaction has inconsistent data.

Simulation: READ UNCOMMITTED

Setup: Bank transfer scenario

```sql
-- Initial state
CREATE TABLE accounts (
    id INT PRIMARY KEY,
    account_holder VARCHAR(100),
    balance DECIMAL(10, 2)
);

INSERT INTO accounts VALUES
(1, 'Alice', 1000.00),
(2, 'Bob', 500.00);
```

Terminal 1 (Transaction A):

```sql
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;

-- Transfer 400 from Alice to Bob (incorrect logic, will rollback)
UPDATE accounts SET balance = balance - 400 WHERE id = 1;
UPDATE accounts SET balance = balance + 400 WHERE id = 2;

-- Simulate delay before rollback
SLEEP(5);

ROLLBACK;
```

Terminal 2 (Transaction B):

```sql
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;

-- Wait 1 second, then read (while Transaction A is still running)
SELECT SLEEP(1);

-- DIRTY READ - reads uncommitted data
SELECT balance FROM accounts WHERE id = 1; -- Shows 600 (WRONG!)
SELECT balance FROM accounts WHERE id = 2; -- Shows 900 (WRONG!)

-- Both show transaction A's changes even though A will rollback

COMMIT;
```

Result:

```
After Transaction B reads (dirty):
- Alice: 600 (should still be 1000)
- Bob: 900 (should still be 500)

After Transaction A rollbacks:
- Alice: 1000 (actual)
- Bob: 500 (actual)
```

Problem: Transaction B saw data that never actually committed. Bob's balance report would be incorrect.

Probable Fix

```sql
-- Option 1: Use READ COMMITTED or higher
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- Option 2: Add application-level validation
BEGIN;
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;  -- Lock the row
-- ... do business logic ...
COMMIT;

-- Option 3: Use explicit locks
LOCK TABLES accounts WRITE;
-- ... do work ...
UNLOCK TABLES;
```

Do It: Fix Implementation

```sql
-- Create a transaction-safe transfer procedure
DELIMITER $$

CREATE PROCEDURE transfer_funds(
    IN from_account_id INT,
    IN to_account_id INT,
    IN amount DECIMAL(10, 2),
    OUT result VARCHAR(100)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET result = 'Transfer failed';
    END;

    START TRANSACTION;
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    -- Lock both accounts
    SELECT balance FROM accounts WHERE id = from_account_id FOR UPDATE;
    SELECT balance FROM accounts WHERE id = to_account_id FOR UPDATE;

    -- Check sufficient funds
    IF (SELECT balance FROM accounts WHERE id = from_account_id) < amount THEN
        ROLLBACK;
        SET result = 'Insufficient funds';
    ELSE
        UPDATE accounts SET balance = balance - amount WHERE id = from_account_id;
        UPDATE accounts SET balance = balance + amount WHERE id = to_account_id;
        COMMIT;
        SET result = 'Transfer successful';
    END IF;
END$$

DELIMITER ;

-- Use it
CALL transfer_funds(1, 2, 100, @result);
SELECT @result;
```

Non-Repeatable Reads

Problem Definition

A **non-repeatable read** (fuzzy read) occurs when a transaction reads the same row twice and gets different values because another transaction modified it in between.

Simulation: READ COMMITTED

Setup: Product stock scenario

```sql
CREATE TABLE products (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    stock INT
);

INSERT INTO products VALUES
(1, 'Laptop', 100),
(2, 'Mouse', 500);
```

Terminal 1 (Transaction A):

```sql
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

-- Read stock first time
SELECT stock FROM products WHERE id = 1; -- Shows 100

-- Simulate delay
SLEEP(3);

-- Read same row again
SELECT stock FROM products WHERE id = 1; -- Shows 90 (CHANGED!)

COMMIT;
```

Terminal 2 (Transaction B):

```sql
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

-- Wait 1 second
SELECT SLEEP(1);

-- Modify stock while Transaction A is still running
UPDATE products SET stock = stock - 10 WHERE id = 1;

COMMIT;
```

Result:

```
First read in A: 100
Transaction B updates: 100 -> 90
Second read in A: 90

Same data read twice, different values!
```

Problem: Inconsistent view of stock levels. Application logic that depends on stock quantity becomes unreliable.

Probable Fix

```sql
-- Option 1: Use REPEATABLE READ
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Option 2: Lock the row for consistency
SELECT stock FROM products WHERE id = 1 FOR UPDATE;

-- Option 3: Use SELECT ... IN SHARE MODE for shared lock
SELECT stock FROM products WHERE id = 1 FOR SHARE;

-- Option 4: Read entire result set at once
SELECT * FROM products WHERE id = 1;
-- Don't issue separate queries on same row
```

Do It: Fix Implementation

```sql
-- Correct approach: REPEATABLE READ with snapshot consistency
CREATE PROCEDURE check_and_update_stock(
    IN product_id INT,
    IN quantity INT,
    OUT sufficient INT
)
BEGIN
    DECLARE current_stock INT;

    SET sufficient = 0;

    START TRANSACTION;
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

    -- First read - get current stock
    SELECT stock INTO current_stock FROM products WHERE id = product_id;

    -- Simulate processing delay
    SELECT SLEEP(2);

    -- Second read - will see SAME value (repeatable)
    SELECT stock INTO current_stock FROM products WHERE id = product_id;

    -- Check if we can fulfill order
    IF current_stock >= quantity THEN
        UPDATE products SET stock = stock - quantity WHERE id = product_id;
        SET sufficient = 1;
    END IF;

    COMMIT;
END$$

DELIMITER ;

-- Test it
CALL check_and_update_stock(1, 10, @result);
SELECT @result; -- Reliable result
```

Verification:

```sql
-- Set default isolation level
SET GLOBAL TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Verify it's working
SELECT @@transaction_isolation;
```

Phantom Reads

Problem Definition

A **phantom read** occurs when a transaction executes a range query twice and gets different numbers of rows because another transaction inserted/deleted rows matching the range.

Simulation: READ COMMITTED / REPEATABLE READ

Setup: Order processing scenario

```sql
CREATE TABLE orders (
    id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    amount DECIMAL(10, 2),
    status VARCHAR(20) DEFAULT 'pending'
);

INSERT INTO orders (customer_id, amount) VALUES
(1, 100.00),
(1, 200.00),
(2, 300.00);
```

Terminal 1 (Transaction A) - READ COMMITTED:

```sql
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

-- Count orders for customer 1, first query
SELECT COUNT(*) FROM orders WHERE customer_id = 1; -- Shows 2

SLEEP(2);

-- Count same orders, second query
SELECT COUNT(*) FROM orders WHERE customer_id = 1; -- Shows 3 (PHANTOM!)

COMMIT;
```

Terminal 2 (Transaction B) - READ COMMITTED:

```sql
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

-- Wait 1 second
SELECT SLEEP(1);

-- Insert new order while Transaction A is running
INSERT INTO orders (customer_id, amount) VALUES (1, 150.00);

COMMIT;
```

Result:

```
First query in A: 2 rows
Transaction B inserts: 1 new row
Second query in A: 3 rows (PHANTOM!)

Same range query, different result set size!
```

Problem: Calculations like "total revenue" or "order count" become unreliable. Reports show inconsistent numbers.

Probable Fix for READ COMMITTED

```sql
-- Option 1: Use REPEATABLE READ (InnoDB with gap locks)
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Option 2: Lock the range
SELECT * FROM orders WHERE customer_id = 1 FOR UPDATE;

-- Option 3: Use SELECT ... FOR SHARE with care
SELECT * FROM orders WHERE customer_id = 1 FOR SHARE;

-- Option 4: Use SERIALIZABLE
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Do It: Fix Implementation

```sql
-- Proper phantom read prevention with REPEATABLE READ
DELIMITER $$

CREATE PROCEDURE get_customer_orders_safe(
    IN cust_id INT,
    OUT order_count INT,
    OUT total_amount DECIMAL(10, 2)
)
BEGIN
    SET order_count = 0;
    SET total_amount = 0.00;

    START TRANSACTION;
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

    -- First query with range lock (gap lock in InnoDB)
    SELECT COUNT(*), COALESCE(SUM(amount), 0)
    INTO order_count, total_amount
    FROM orders
    WHERE customer_id = cust_id;

    -- Second query will see SAME rows (no phantoms)
    SELECT COUNT(*) INTO @verify_count
    FROM orders
    WHERE customer_id = cust_id;

    -- Verify consistency
    IF order_count = @verify_count THEN
        COMMIT;
    ELSE
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Phantom read detected!';
    END IF;
END$$

DELIMITER ;

-- Test it
CALL get_customer_orders_safe(1, @cnt, @amt);
SELECT @cnt, @amt;
```

InnoDB-Specific Gap Lock Prevention:

```
-- InnoDB prevents phantoms in REPEATABLE READ using gap locks
SHOW ENGINE INNODB STATUS\G

-- Check lock waits
SELECT * FROM INFORMATION_SCHEMA.INNODB_LOCKS;
SELECT * FROM INFORMATION_SCHEMA.INNODB_LOCK_WAITS;
```

Lost Updates

Problem Definition

A **lost update** occurs when two transactions read the same value, both modify it, and both commit. One transaction's update is lost.

Simulation: All Isolation Levels (except SERIALIZABLE)

Setup: Counter/Balance scenario

```
CREATE TABLE counter (
    id INT PRIMARY KEY,
    value INT
);

INSERT INTO counter VALUES (1, 0);
```

Terminal 1 (Transaction A):

```sql
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

-- Read current value
SELECT value FROM counter WHERE id = 1; -- Shows 0

-- Simulate processing
SELECT SLEEP(2);

-- Increment and save
UPDATE counter SET value = value + 5 WHERE id = 1;

COMMIT;
```

Terminal 2 (Transaction B) - Start simultaneously:

```sql
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

-- Read current value (sees same initial value as A)
SELECT value FROM counter WHERE id = 1; -- Shows 0

-- Simulate processing
SELECT SLEEP(1);

-- Increment and save
UPDATE counter SET value = value + 10 WHERE id = 1;

COMMIT;
```

Sequence of Events:

```
Time 1: A reads: 0
Time 1: B reads: 0
Time 2: B updates: 0 + 10 = 10, commits
Time 3: A updates: 0 + 5 = 5, commits (overwrites B's change!)

Final result: 5 (should be 15!)
```

Problem: Both increments should have accumulated, but one is lost.

Probable Fix

```sql
-- Option 1: Use atomic UPDATE instead of SELECT + UPDATE
UPDATE counter SET value = value + 5 WHERE id = 1;

-- Option 2: Use explicit locking
SELECT value FROM counter WHERE id = 1 FOR UPDATE;
-- ... do logic ...
UPDATE counter SET value = ... WHERE id = 1;

-- Option 3: Use SERIALIZABLE
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Option 4: Application-level retry logic with versioning
```

Do It: Fix Implementation

```sql
-- Solution 1: Atomic operations (BEST)
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Single atomic operation - no lost updates
UPDATE counter SET value = value + 5 WHERE id = 1;

COMMIT;

-- Solution 2: WITH optimistic locking (version column)
ALTER TABLE counter ADD COLUMN version INT DEFAULT 0;

DELIMITER $$

CREATE PROCEDURE increment_counter_safe(
    IN increment_val INT,
    OUT success INT
)
BEGIN
    DECLARE current_version INT;
    DECLARE current_value INT;

    SET success = 0;

    START TRANSACTION;
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

    -- Read current value and version
    SELECT value, version
    INTO current_value, current_version
    FROM counter
    WHERE id = 1 FOR UPDATE;  -- Explicit lock

    -- Update with version check
    UPDATE counter
    SET value = current_value + increment_val,
        version = version + 1
    WHERE id = 1 AND version = current_version;

    IF ROW_COUNT() = 0 THEN
        ROLLBACK;
```

```sql
        SET success = 0;  -- Conflict detected
    ELSE
        COMMIT;
        SET success = 1;  -- Success
    END IF;
END$$

DELIMITER ;

-- Test it
CALL increment_counter_safe(5, @success);
SELECT @success;

-- Solution 3: Use pessimistic locking
DELIMITER $$

CREATE PROCEDURE increment_counter_lock(
    IN increment_val INT
)
BEGIN
    START TRANSACTION;
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

    -- Lock immediately - prevents lost updates
    SELECT value FROM counter WHERE id = 1 FOR UPDATE;

    -- Safe to update
    UPDATE counter SET value = value + increment_val WHERE id = 1;

    COMMIT;
END$$

DELIMITER ;

CALL increment_counter_lock(5);
```

Implementation & Examples

Checking Current Isolation Level

```sql
-- Session level
SELECT @@transaction_isolation;
SELECT @@SESSION.transaction_isolation;

-- Global level
SELECT @@GLOBAL.transaction_isolation;

-- All session variables
SHOW VARIABLES LIKE '%isolation%';
```

Setting Isolation Level

```sql
-- Next transaction only
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN;
-- ... work ...
COMMIT;

-- Entire session
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Global (all new connections)
SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- In connection string
mysql --default-transaction-isolation=READ-COMMITTED
```

Permanent Configuration

```
# my.cnf or my.ini
[mysqld]
transaction-isolation = REPEATABLE-READ
# OR
transaction-isolation = READ-COMMITTED
# OR
transaction-isolation = SERIALIZABLE
# OR
transaction-isolation = READ-UNCOMMITTED
```

Real-World Scenarios

Scenario 1: E-Commerce Inventory

```sql
-- Problem: Overselling due to non-repeatable reads
CREATE TABLE inventory (
    product_id INT PRIMARY KEY,
    quantity INT,
    reserved INT DEFAULT 0
);

INSERT INTO inventory VALUES (1, 100, 0);

-- WRONG APPROACH (READ COMMITTED)
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT quantity INTO @qty FROM inventory WHERE product_id = 1;
-- Another process decreases quantity here!
UPDATE inventory SET reserved = reserved + 50 WHERE product_id = 1 AND quantity >= 50;

COMMIT;

-- CORRECT APPROACH (with locking)
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Lock the row immediately
SELECT quantity INTO @qty FROM inventory WHERE product_id = 1 FOR UPDATE;

-- Now safe - quantity is locked
IF @qty >= 50 THEN
    UPDATE inventory SET reserved = reserved + 50 WHERE product_id = 1;
ELSE
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Not enough stock';
END IF;

COMMIT;
```

Scenario 2: Financial Calculations

```sql
-- Problem: Phantom reads in account reconciliation
CREATE TABLE transactions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT,
    amount DECIMAL(10, 2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- WRONG (READ COMMITTED) - phantom reads possible
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- Sum transactions for reconciliation
SELECT SUM(amount) INTO @total
FROM transactions
WHERE account_id = 1 AND DATE(created_at) = CURDATE();

-- Another process adds transaction here!
SELECT COUNT(*) INTO @count
FROM transactions
WHERE account_id = 1 AND DATE(created_at) = CURDATE();

-- Total and count don't match!

-- CORRECT (REPEATABLE READ or SERIALIZABLE)
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

SELECT SUM(amount) INTO @total, COUNT(*) INTO @count
FROM transactions
WHERE account_id = 1 AND DATE(created_at) = CURDATE();

-- No phantoms - total and count are consistent
INSERT INTO audit_log VALUES (1, @total, @count, NOW());

COMMIT;
```

Scenario 3: Update with Checks

```sql
-- Problem: Lost updates in approval workflow
CREATE TABLE expense_requests (
    id INT PRIMARY KEY,
    status VARCHAR(20),
    amount DECIMAL(10, 2),
    approver_id INT
);

INSERT INTO expense_requests VALUES (1, 'pending', 500.00, NULL);

-- WRONG (can lose approval)
START TRANSACTION;
IF EXISTS(SELECT 1 FROM expense_requests WHERE id = 1 AND status = 'pending') THEN
    UPDATE expense_requests SET status = 'approved', approver_id = 123 WHERE id = 1;
END IF;
COMMIT;

-- CORRECT (atomic operation)
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

UPDATE expense_requests
SET status = 'approved', approver_id = 123
WHERE id = 1 AND status = 'pending';

IF ROW_COUNT() = 0 THEN
    -- Status was already changed by another process
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request already processed';
ELSE
    COMMIT;
END IF;
```

Monitoring & Troubleshooting

View Active Transactions

```sql
-- InnoDB transactions
SELECT * FROM INFORMATION_SCHEMA.INNODB_TRX;

-- Detailed format
SELECT
    trx_id,
    trx_requested_lock_id,
    trx_mysql_thread_id,
    trx_state,
    trx_started,
    trx_wait_started,
    trx_isolation_level
FROM INFORMATION_SCHEMA.INNODB_TRX\G

-- Running processes
SHOW PROCESSLIST;

-- Detailed process list
SELECT * FROM INFORMATION_SCHEMA.PROCESSLIST;
```

Identify Locks and Waits

```sql
-- Current locks
SELECT * FROM INFORMATION_SCHEMA.INNODB_LOCKS;

-- Lock waits
SELECT * FROM INFORMATION_SCHEMA.INNODB_LOCK_WAITS;

-- Locks by table
SELECT object_schema, object_name, count(*)
FROM INFORMATION_SCHEMA.INNODB_LOCKS
GROUP BY object_schema, object_name;

-- Who's waiting on whom
SELECT waiting_trx_id, waiting_pid, waiting_query,
       blocking_trx_id, blocking_pid, blocking_query
FROM sys.innodb_lock_waits;
```

Deadlock Detection

```sql
-- Enable deadlock logging
SET GLOBAL innodb_print_all_deadlocks = ON;

-- Monitor deadlocks
SHOW ENGINE INNODB STATUS\G

-- Check error log
TAIL -f /var/log/mysql/error.log

-- Application level retry logic
DELIMITER $$

CREATE PROCEDURE retry_transaction(
    IN max_retries INT = 5
)
BEGIN
    DECLARE retry_count INT DEFAULT 0;
    DECLARE success INT DEFAULT 0;

    WHILE retry_count < max_retries AND success = 0 DO
        BEGIN
            DECLARE CONTINUE HANDLER FOR SQLSTATE '40001' BEGIN END;

            START TRANSACTION;
            SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

            -- Your transaction logic here

            COMMIT;
            SET success = 1;
        END;

        IF success = 0 THEN
            SET retry_count = retry_count + 1;
            SELECT SLEEP(0.1 * retry_count);  -- Exponential backoff
        END IF;
    END WHILE;

    IF success = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction failed after retries';
    END IF;
```

```
END$$

DELIMITER ;
```

Performance Impact Analysis

```sql
-- Compare isolation levels performance
-- Create test table
CREATE TABLE perf_test (
    id INT PRIMARY KEY,
    value INT,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Benchmark script
DELIMITER $$

CREATE PROCEDURE benchmark_isolation(
    IN isolation_level VARCHAR(50),
    IN num_iterations INT = 1000
)
BEGIN
    DECLARE i INT DEFAULT 0;
    DECLARE start_time DATETIME;
    DECLARE end_time DATETIME;
    DECLARE elapsed_seconds DECIMAL(10, 3);

    SET start_time = NOW(6);

    SET SESSION TRANSACTION ISOLATION LEVEL = isolation_level;

    WHILE i < num_iterations DO
        START TRANSACTION;

        UPDATE perf_test SET value = value + 1 WHERE id = 1;

        COMMIT;
        SET i = i + 1;
    END WHILE;

    SET end_time = NOW(6);
    SET elapsed_seconds = TIMESTAMPDIFF(MICROSECOND, start_time, end_time) / 1000000;

    SELECT isolation_level, num_iterations, elapsed_seconds;
END$$

DELIMITER ;
```

```sql
-- Run benchmarks
CALL benchmark_isolation('READ UNCOMMITTED', 1000);
CALL benchmark_isolation('READ COMMITTED', 1000);
CALL benchmark_isolation('REPEATABLE READ', 1000);
CALL benchmark_isolation('SERIALIZABLE', 1000);
```

Best Practices

1. Choose Right Isolation Level

```sql
-- For most applications: REPEATABLE READ (MySQL default)
-- Balances safety with performance
-- InnoDB gap locks prevent phantoms

-- For financial systems: SERIALIZABLE
-- Use only for critical operations
-- Implement retry logic for conflicts

-- For read-heavy systems: READ COMMITTED
-- If you can handle non-repeatable reads
-- Oracle database default

-- Almost never: READ UNCOMMITTED
-- No real use case in production
-- Safety risks outweigh performance gains
```

2. Minimize Transaction Duration

```sql
-- WRONG: Long transaction
START TRANSACTION;
SELECT ... (complex query);
SLEEP(5);  -- Some processing
INSERT ... (multiple statements);
UPDATE ... (multiple statements);
COMMIT;

-- CORRECT: Short transaction
SELECT ... (fetch needed data);
-- Do processing outside transaction
START TRANSACTION;
INSERT ... (single operation);
COMMIT;
```

3. Use Explicit Locking When Needed

```sql
-- For critical sections, use FOR UPDATE
START TRANSACTION;

-- Lock prevents other modifications
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;

-- Safe to update after checking
UPDATE accounts SET balance = balance + 100 WHERE id = 1;

COMMIT;

-- FOR SHARE for read locks
SELECT balance FROM accounts WHERE id = 1 FOR SHARE;
```

4. Handle Deadlocks in Application

```java
// Java example - retry logic for deadlocks
int maxRetries = 3;
int retryCount = 0;
boolean success = false;

while (retryCount < maxRetries && !success) {
    try {
        connection.setAutoCommit(false);
        // ... perform database operations ...
        connection.commit();
        success = true;
    } catch (SQLException e) {
        if ("40001".equals(e.getSQLState())) {  // Deadlock
            retryCount++;
            if (retryCount >= maxRetries) throw e;
            Thread.sleep((long) Math.pow(2, retryCount) * 100);  // Exponential backoff
        } else {
            throw e;
        }
    } finally {
        if (!success) connection.rollback();
    }
}
```

5. Monitor Production Systems

```sql
-- Regular monitoring queries
-- Set up alerts if these exceed thresholds

-- Active transactions
SELECT COUNT(*) as active_transactions
FROM INFORMATION_SCHEMA.INNODB_TRX;

-- Transactions waiting on locks
SELECT COUNT(*) as waiting_transactions
FROM INFORMATION_SCHEMA.INNODB_LOCK_WAITS;

-- Long-running transactions
SELECT
    trx_id,
    TIMESTAMPDIFF(SECOND, trx_started, NOW()) as running_seconds,
    trx_state
FROM INFORMATION_SCHEMA.INNODB_TRX
WHERE TIMESTAMPDIFF(SECOND, trx_started, NOW()) > 60;

-- Monitoring script
-- Create this event for continuous monitoring
CREATE EVENT monitor_transactions
ON SCHEDULE EVERY 1 MINUTE
DO
BEGIN
    INSERT INTO monitoring_log
    SELECT
        NOW(),
        COUNT(*),
        TIMESTAMPDIFF(SECOND, MIN(trx_started), NOW())
    FROM INFORMATION_SCHEMA.INNODB_TRX;
END;
```

6. Use Connection Pooling

```
-- Connection pool settings
-- Each connection can maintain its own isolation level
-- Avoid changing global settings at runtime

-- In application config:
# hikaricp properties
hikaricp.jdbc.url = jdbc:mysql://localhost:3306/db
hikaricp.jdbc.user = user
hikaricp.jdbc.password = pass
hikaricp.connectionInitSql = SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ
hikaricp.maximumPoolSize = 20
```

7. Document Transaction Requirements

```sql
-- Always document isolation level requirements in stored procedures
DELIMITER $$

/**
 * Transfer funds between accounts
 *
 * ISOLATION LEVEL: REPEATABLE READ
 * REASON: Prevents non-repeatable reads and lost updates
 *         ensures accurate balance calculations
 *
 * LOCKS: Both accounts are locked via FOR UPDATE
 * DURATION: < 1 second expected
 */
CREATE PROCEDURE transfer_funds(
    IN from_id INT,
    IN to_id INT,
    IN amount DECIMAL(10, 2)
)
BEGIN
    START TRANSACTION;
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

    -- Documentation for future maintainers
    -- Lock in consistent order (by id) to prevent circular deadlocks
    SELECT balance FROM accounts WHERE id = from_id FOR UPDATE;
    SELECT balance FROM accounts WHERE id = to_id FOR UPDATE;

    -- ... transfer logic ...

    COMMIT;
END$$

DELIMITER ;
```

Quick Reference

When to Use Each Level

Level	Use Case	Trade-off
READ UNCOMMITTED	Never	Unreliable data
READ COMMITTED	Read-heavy systems, Oracle compat	Non-repeatable reads
REPEATABLE READ	Most applications (MySQL default)	Slight phantom read risk
SERIALIZABLE	Financial operations, critical sections	High performance impact

Common Issues & Solutions

Issue	Cause	Solution
Slow queries	Locks held too long	Reduce transaction duration
Deadlocks	Circular lock waits	Lock in consistent order, use retries
Inconsistent data	Insufficient isolation	Increase isolation level, use explicit locks
Lost updates	Non-atomic operations	Use atomic UPDATE, versioning
High latency	Contention	Review transaction design

Performance Guidelines

```
READ UNCOMMITTED    - Fastest, unreliable
READ COMMITTED      - Fast, some anomalies
REPEATABLE READ     - Normal (default), safe
SERIALIZABLE        - Slow, safest
```

MySQL-Specific Notes

1. **InnoDB (default engine)**:
    - Supports all 4 isolation levels
    - Uses MVCC (Multi-Version Concurrency Control)
    - Gap locks prevent phantoms in REPEATABLE READ

    - Default is REPEATABLE READ
2. **MyISAM (table-level locks)**:
    - Doesn't truly implement isolation levels
    - Table-level locking, not row-level
    - Avoid for concurrent workloads
3. **Transaction options**:
    - `AUTOCOMMIT = 0` for implicit transactions
    - `START TRANSACTION` for explicit transactions
    - `SAVEPOINT` for nested rollback capability

Summary

Isolation levels are critical for database correctness and consistency:

- **READ UNCOMMITTED**: Avoid in production
- **READ COMMITTED**: Read-heavy systems, trade some safety
- **REPEATABLE READ**: MySQL default, recommended for most apps
- **SERIALIZABLE**: Only for critical operations

Key problems to prevent:

1. Dirty reads → use READ COMMITTED or higher
2. Non-repeatable reads → use REPEATABLE READ or higher
3. Phantom reads → use SERIALIZABLE or gap locks
4. Lost updates → use explicit locking or atomic operations

Production checklist:

- Set appropriate isolation level for your use case
- Implement explicit locks for critical sections
- Keep transactions short
- Handle deadlocks with retry logic
- Monitor active transactions and lock waits
- Document isolation requirements in code
- Test under load with concurrent scenarios